

Document number: P1050R1

Date: 2019-06-17

Reply-to: John McFarlane, fixed-point@john.mcfarlane.name

Audience: SG6, LEWG

Fractional Numeric Type

Abstract

This paper introduces a fractional number type which stores pairs of integer numerators and denominators. It avoids precision loss commonly associated with the storage of single-number quotients. It also helps express intent when initializing fixed-point types with the results of division operations.

Introduction

Rational numbers provide a headache for digital number types such as the fundamental scalars. We already have `std::ratio` -- mostly as a necessary way of expressing different static scales in `std::chrono`. There has previously been discussion of sophisticated, general purpose bounded [P0101] and unbounded [P0101] rationals. [P1438] concentrates on unbounded rationals. Boost has a rational type [Boost].

This paper proposes a fractional type that is compatible with the compositional approach detailed in [P0554]. As such it is designed to provide a zero-overhead abstraction over pairs of integers while also being an expressive, user-friendly, general-purpose statically-sized number type when combined with other numeric components.

The reason for proposing a fractional type at this time is because it serves a specific role in initialization of the fixed-point numbers described in [P0037].

Motivation

Class template, `fixed_point<>` from [P0037] can be initialized with integer and floating-point values. It is desirable that `fixed_point<>` values also be the quotient in a division operation. A `divide` function was previously proposed in [P0037]:

```
auto a = divide(1, 3);
```

Straight away the question arises: what should be the resolution of `a`? The solution proposed in [P0106] is to determine the number of fractional digits from the number of fractional digits in the numerator and integer digits in the denominator. In the example above, `a` has 31 fractional digits because literal, `3`, has 31 integer digits. The result has value, 0.333333333022892475128173828125.

This solution is problematic when one considers all of the ways that `1` or `3` can be represented. For example, using `elastic_integer<>` [P0828] as the inputs

```
auto b = divide(elastic_integer<1>{1}, elastic_integer<2>{3});
```

results in a `fixed_point` type with only 2 fractional digits. The resultant approximation of `1/3` is `0.25`. This is unlikely to be the desired result in most cases.

Specifying the output type is one solution:

```
auto c = divide<fixed_point<int, -16>>(1, 3);  
auto d = divide<fixed_point<int, -16>>(elastic_integer<1>{1},  
elastic_integer<2>{3});
```

Both `c` and `d` have values, `0.3333282470703125`. However, it is not entirely clear that introducing a named function specifically for this purpose is necessary, nor whether the interface is sufficiently intuitive.

By introducing a fractional type, `fraction`

```
template<class Numerator, class Denominator>  
struct fraction {  
    // ...  
  
    Numerator numerator;  
    Denominator denominator;  
};
```

we can express the same intent using only types:

```
auto a = fixed_point{fraction{1, 3}};  
auto b = fixed_point{fraction{elastic_integer<1>{1}, elastic_integer<2>  
{3}}};  
auto c = fixed_point<int, -16>{fraction{1, 3}};  
auto d = fixed_point<int, -16>{fraction{elastic_integer<1>{1},  
elastic_integer<2>{3}}};
```

Overflow, Reduction and Normalization

A well understood problem with fraction arithmetic is that successive operations often cause an exponential increase in the magnitude of both the numerator and denominator.

There is no silver bullet which solves this problem, other than making both numbers unbounded. Mitigations include down-scaling, which loses precision and reduction (`-3/-9` $\rightarrow$ `-1/-3`) or normalization (`-3/-9` $\rightarrow$ `1/3`), which are more costly than the operations themselves. Reduction or normalization can be applied lazily or periodically to reduce the cost but no single strategy is optimal in all cases. This is a dilemma which must be exposed to the user. Otherwise they will implement their own, lower-level solutions.

`fraction` is designed to support every conceivable approach to solving this dilemma. Some solutions involve replacing fundamental integers with types which detect or avoid overflow. Others involve using `fractional` as the basis for a type with more complex behavior. Examples follow...

Don't Overflow

The zero-cost solution is to ensure that overflow never occurs:

```
fraction a{2, 3};
auto b{a*a};
```

[\[example\]](#)

A 'safe integer' type or a sanitizer can be used to ensure correctness:

```
fraction<overflow_integer<int>> c{1, 1'000'000'000};
auto d{c*c}; // traps
```

[\[example\]](#)

An auto-widening type avoids overflow:

```
fraction g{1_elastic, 3_elastic};

// fraction with a 1-bit denominator and a 4-bit numerator
auto h{g*g};
```

[\[example\]](#)

Eager Reduction

Reduction can mitigate the risk of overflow:

```
fraction i{25, 64};
fraction j{12, 3};
```

```
auto k{reduce(i*j)};
cout << k << '\n';
```

[example]

Normalization works just as well. The nice thing about normalization is that it simplifies comparison. But a class is required to maintain the invariant:

```
class rational
{
private:
    fraction<> f;
public:
    friend auto operator==(rational a, rational b) {
        // optimisation: denominators do not need to be normalised before
        comparison
        return std::tie(a.f.numerator, a.f.denominator)
            == std::tie(b.f.numerator, b.f.denominator);
    }
};
```

Lazy Reduction

The following wrapper reduces the fraction in reaction to an overflow exception.

```
template<int Digits>
using safe_integer = overflow_integer<elastic_integer<Digits>,
    throwing_overflow_tag>;

class rational
{
private:
    using fract = cnl::fraction<safe_integer<31>>;
    fract f;
public:
    friend auto operator*(rational a, rational b) {
        auto prod{a.f*a.f}; // 62 digit integers
        try {
            // narrow to 31 digits; throw on overflow
            return rational(prod);
        }
    }
};
```

```

    catch(std::overflow_error const&) {
        // Reduce and try again to narrow.
        // Overflow may still happen but hey, we tried.
        return rational(fract(reduce(prod)));
    }
}
};

```

[\[example\]](#)

Synopsis

```

template<class Numerator, class Denominator>
struct fraction {
    using numerator_type = Numerator;
    using denominator_type = Denominator;

    explicit constexpr fraction(const Numerator& n, const Denominator& d);
    explicit constexpr fraction(const Numerator& n);

    template<class Scalar,
    _impl::enable_if_t<std::is_floating_point<Scalar>::value, int> = 0>
    explicit constexpr operator Scalar() const;

    numerator_type numerator;
    denominator_type denominator = 1;
};

template<class Numerator, class Denominator>
fraction(const Numerator& n, const Denominator&)
-> fraction<Numerator, Denominator>;

template<class Numerator>
fraction(Numerator const& n)
-> fraction<Numerator, int>;

template<class Numerator, class Denominator>
constexpr auto reduce(fraction<Numerator, Denominator> const& f);

template<class LhsNumerator, class LhsDenominator, class RhsNumerator,
class RhsDenominator>

```

```

constexpr auto operator+(
    fraction<LhsNumerator, LhsDenominator> const& lhs,
    fraction<RhsNumerator, RhsDenominator> const& rhs)
-> decltype(make_fraction(
    lhs.numerator*rhs.denominator+rhs.numerator*lhs.denominator,
    lhs.denominator*rhs.denominator));

template<class LhsNumerator, class LhsDenominator, class RhsNumerator,
class RhsDenominator>
constexpr auto operator-(
    fraction<LhsNumerator, LhsDenominator> const& lhs,
    fraction<RhsNumerator, RhsDenominator> const& rhs)
-> decltype(make_fraction(
    lhs.numerator*rhs.denominator-rhs.numerator*lhs.denominator,
    lhs.denominator*rhs.denominator));

template<class LhsNumerator, class LhsDenominator, class RhsNumerator,
class RhsDenominator>
constexpr auto operator*(
    fraction<LhsNumerator, LhsDenominator> const& lhs,
    fraction<RhsNumerator, RhsDenominator> const& rhs)
-> decltype(make_fraction(lhs.numerator*rhs.numerator,
    lhs.denominator*rhs.denominator));

template<class LhsNumerator, class LhsDenominator, class RhsNumerator,
class RhsDenominator>
constexpr auto operator/(
    fraction<LhsNumerator, LhsDenominator> const& lhs,
    fraction<RhsNumerator, RhsDenominator> const& rhs)
-> decltype(make_fraction(lhs.numerator*rhs.denominator,
    lhs.denominator*rhs.numerator));

template<class LhsNumerator, class LhsDenominator, class RhsNumerator,
class RhsDenominator>
constexpr auto operator==(
    fraction<LhsNumerator, LhsDenominator> const& lhs,
    fraction<RhsNumerator, RhsDenominator> const& rhs);

template<class LhsNumerator, class LhsDenominator, class RhsNumerator,
class RhsDenominator>

```

```
constexpr auto operator!=(  
    fraction<LhsNumerator, LhsDenominator> const& lhs,  
    fraction<RhsNumerator, RhsDenominator> const& rhs);
```

Reference Implementation

`fraction` is implemented as part of [the CNL library](#) ([header](#), [tests](#), [fixed_point integration](#)).